



Programación

Tema 9: Pruebas con JUnit

¿Te puedes imaginar un mundo sin pruebas?



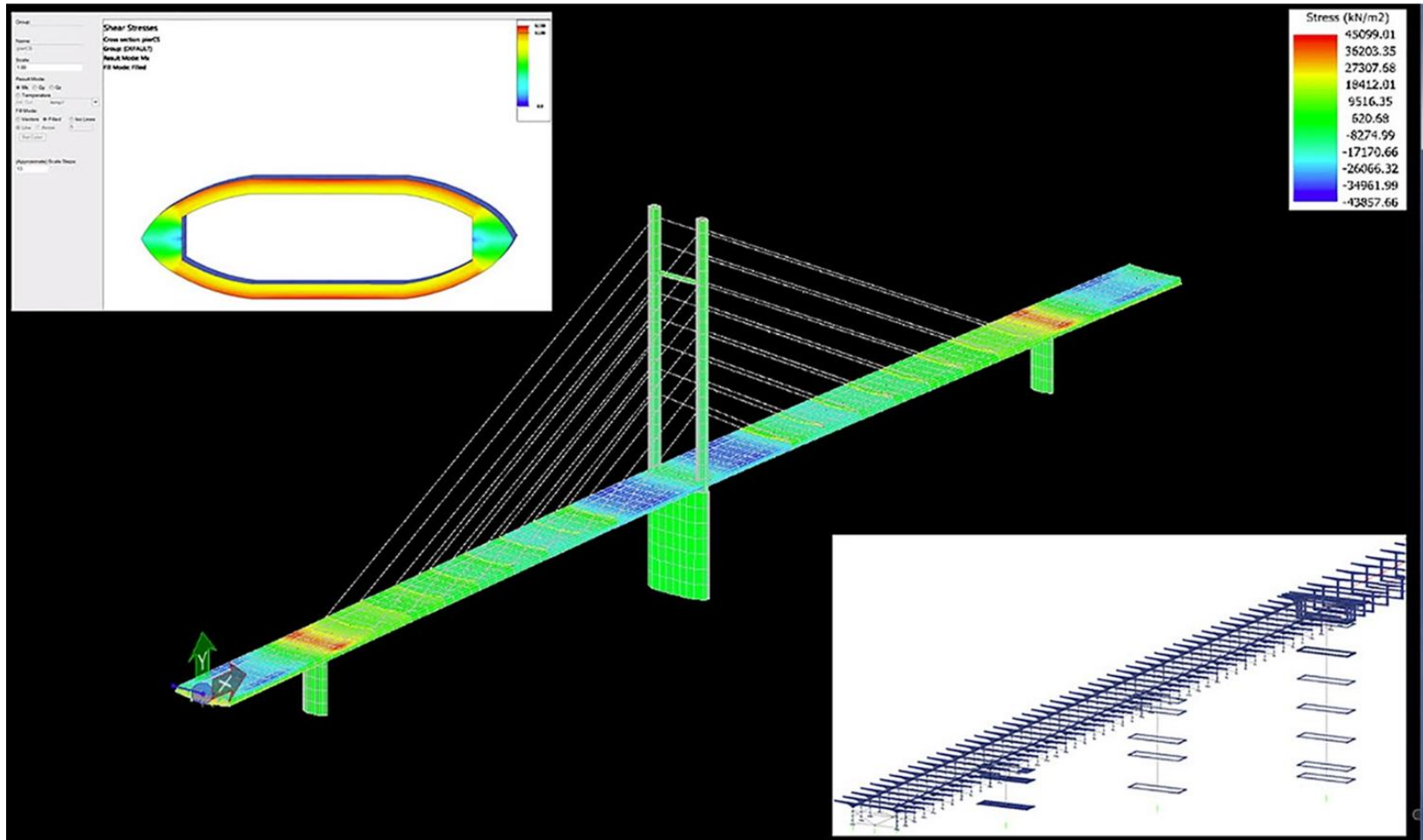
¿Te puedes imaginar un mundo sin pruebas?



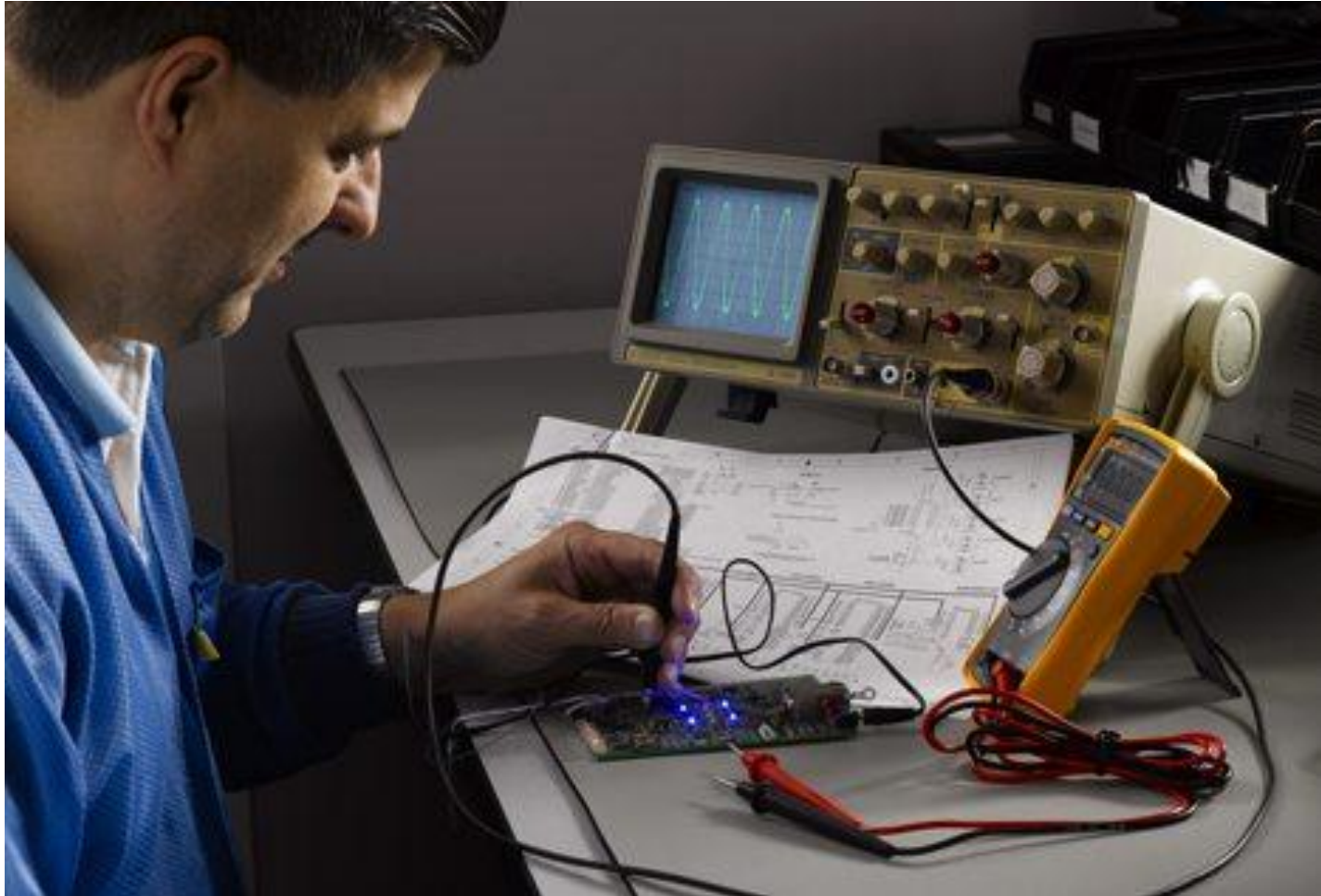
¿Te puedes imaginar un mundo sin pruebas?



¿Te puedes imaginar un mundo sin pruebas?



¿Te puedes imaginar un mundo sin pruebas?



¿En programación es importante?

¿

```
private long doit(Graph graph) {  
    Node node0 = graph.getNode("0");  
    Node node1 = graph.getNode("1");  
    Node node2 = graph.getNode("2");  
    Node node3 = graph.getNode("3");  
    Node node4 = graph.getNode("4");  
    Node node5 = graph.getNode("5");  
    BFS bfs = new BFS(graph);  
    long t0 = System.currentTimeMillis();  
    bfs.search(node0, node1);  
    bfs.search(node0, node2);  
    bfs.search(node0, node3);  
    bfs.search(node0, node4);  
    bfs.search(node0, node5);  
    long t2 = System.currentTimeMillis();  
    return t2 - t0;  
}
```

?

¿En programación es importante?

Etiopía

Un fallo del software ~~THE OBJECTIVE~~ del Boeing 737 MAX 8 causó el accidente de Ethiopian Airlines

4 de abril del 2019 • 10:53 am CEST

Última actualización: 4 Abr 2019, 2:45 pm CEST

El **sistema automatizado** de este avión, conocido como MCAS, ha sido objeto de investigaciones de la Justicia estadounidense y estuvo también involucrado en el accidente aéreo de Lion Air en Indonesia con un avión 737 MAX 8 en octubre de 2018, en el que murieron 189 personas. Tras el accidente, numerosos países dejaron de operar con este modelo de avión.



¿En programación es importante?

Un fallo de seguridad en Facebook revela fotos privadas de 7 millones de usuarios

MCPRO

Publicado el 15 diciembre, 2018 por [Juan Ranchal](#) 

Toyota llama a revisión a 2,43 millones de coches híbridos por un fallo de software: hay 28.860 afectados en España

elEconomista.es

Europa Press / Reuters

¿En programación es importante?



¿En programación es importante?

Un problema en marcapasos Medtronic urge a reprogramarlos en modo a prueba de fallos

04/02/2019 18:52 - ACTUALIZADO: 04/02/2019 22:08

El Confidencial

Un error de software deja para desguace 300 coches de Subaru



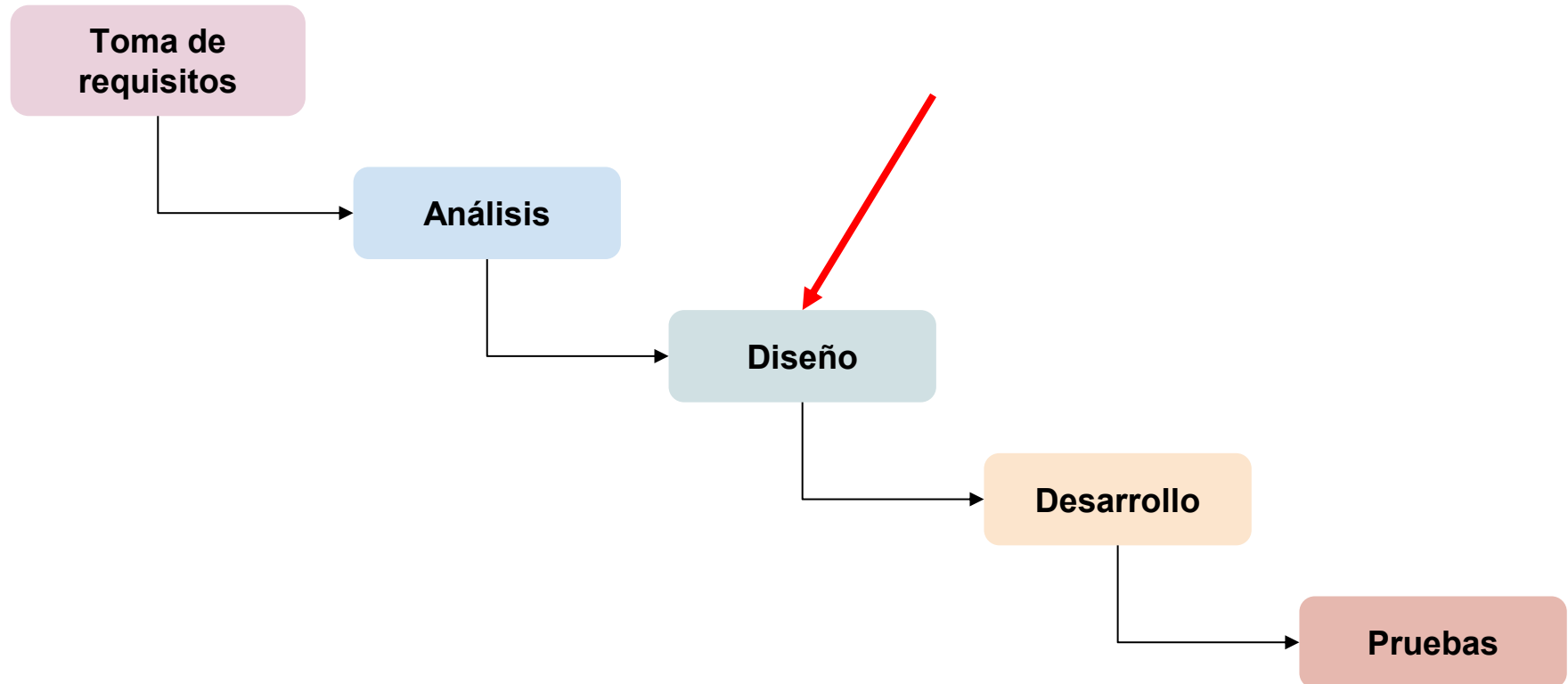
Más de 60 vuelos cancelados en Frankfurt por un fallo de software: Lufthansa, la más afectada con 46

FRANKFURT, 25 Mar. (EUROPA PRESS)

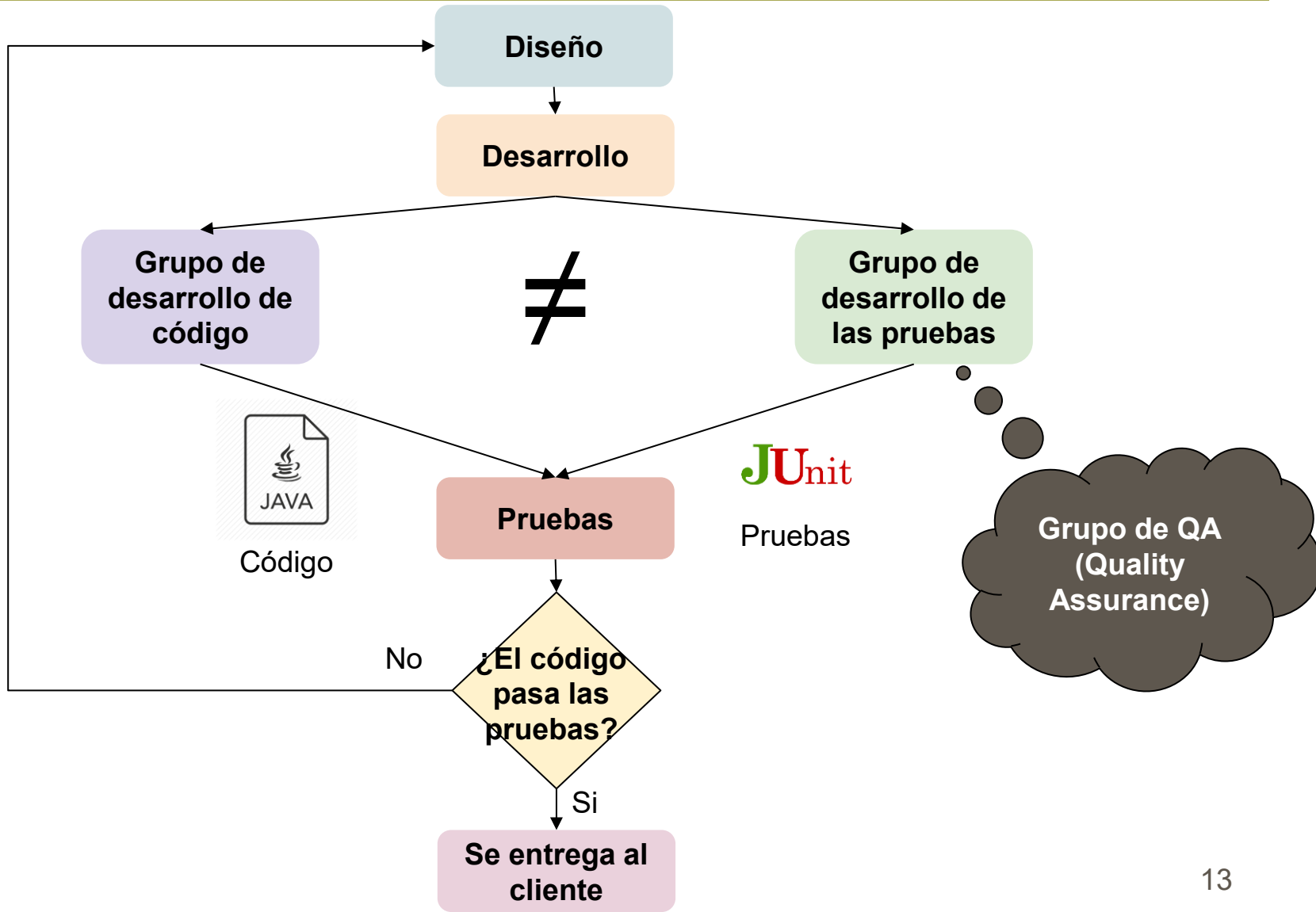
europa press

Ciclo de desarrollo software

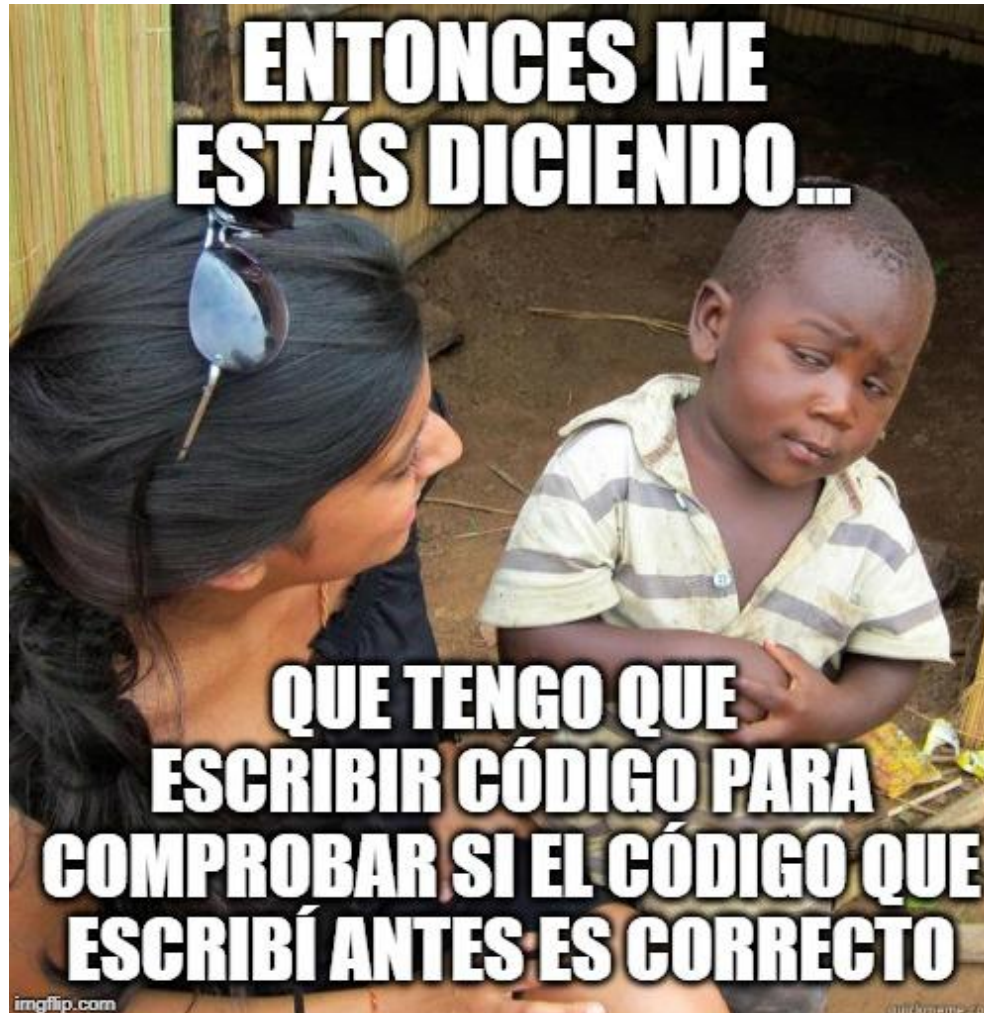
¿Cuándo diseñar las pruebas?



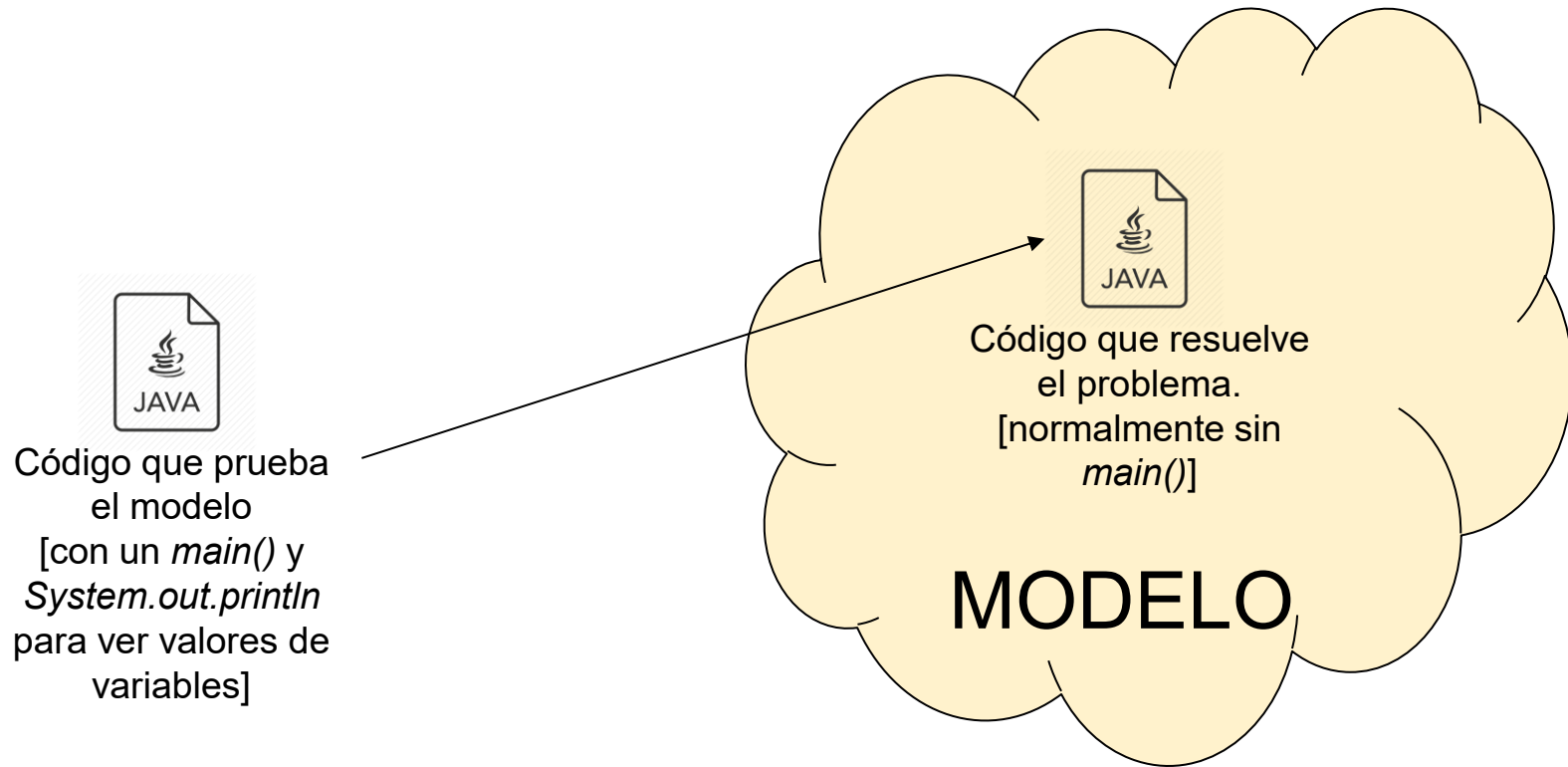
¿Quién debería hacer las pruebas?



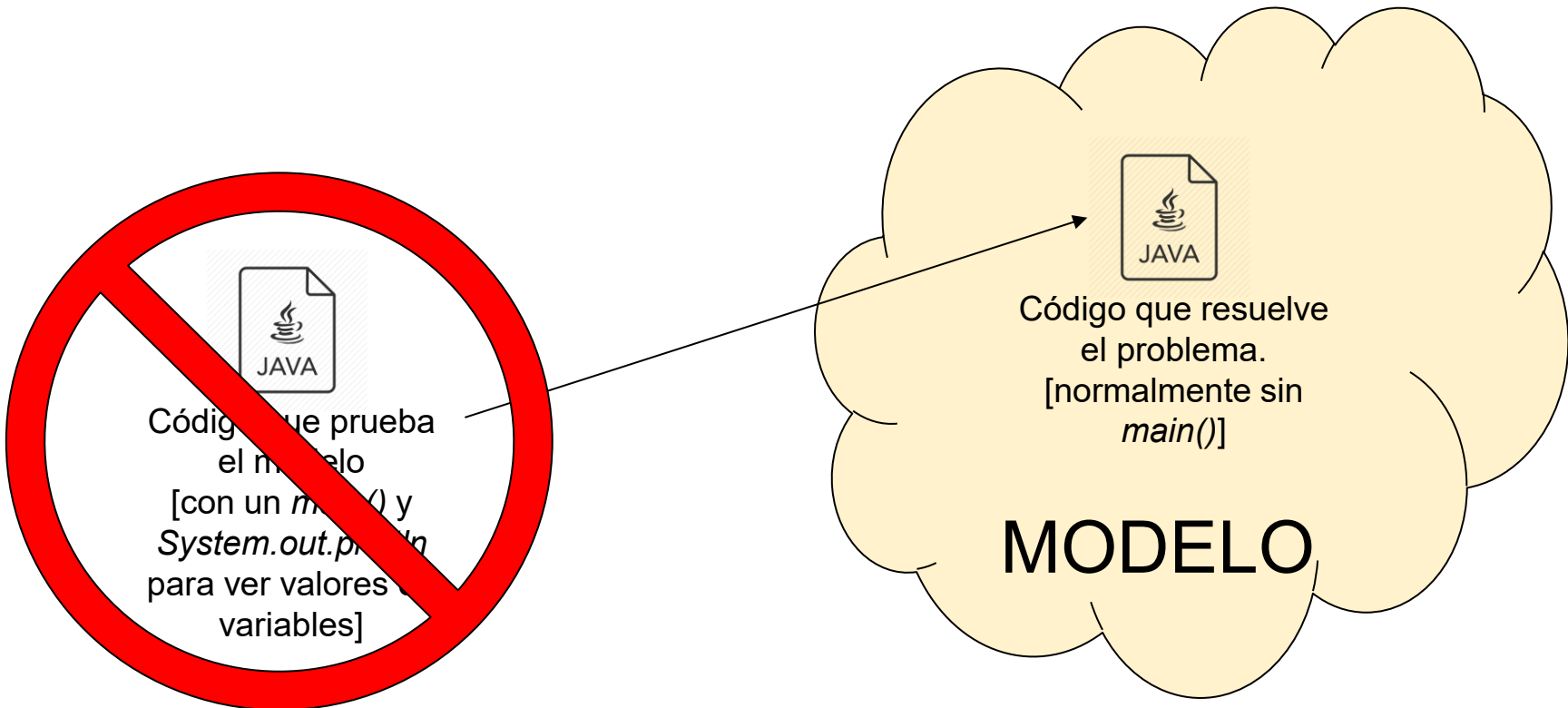
Calidad del Software



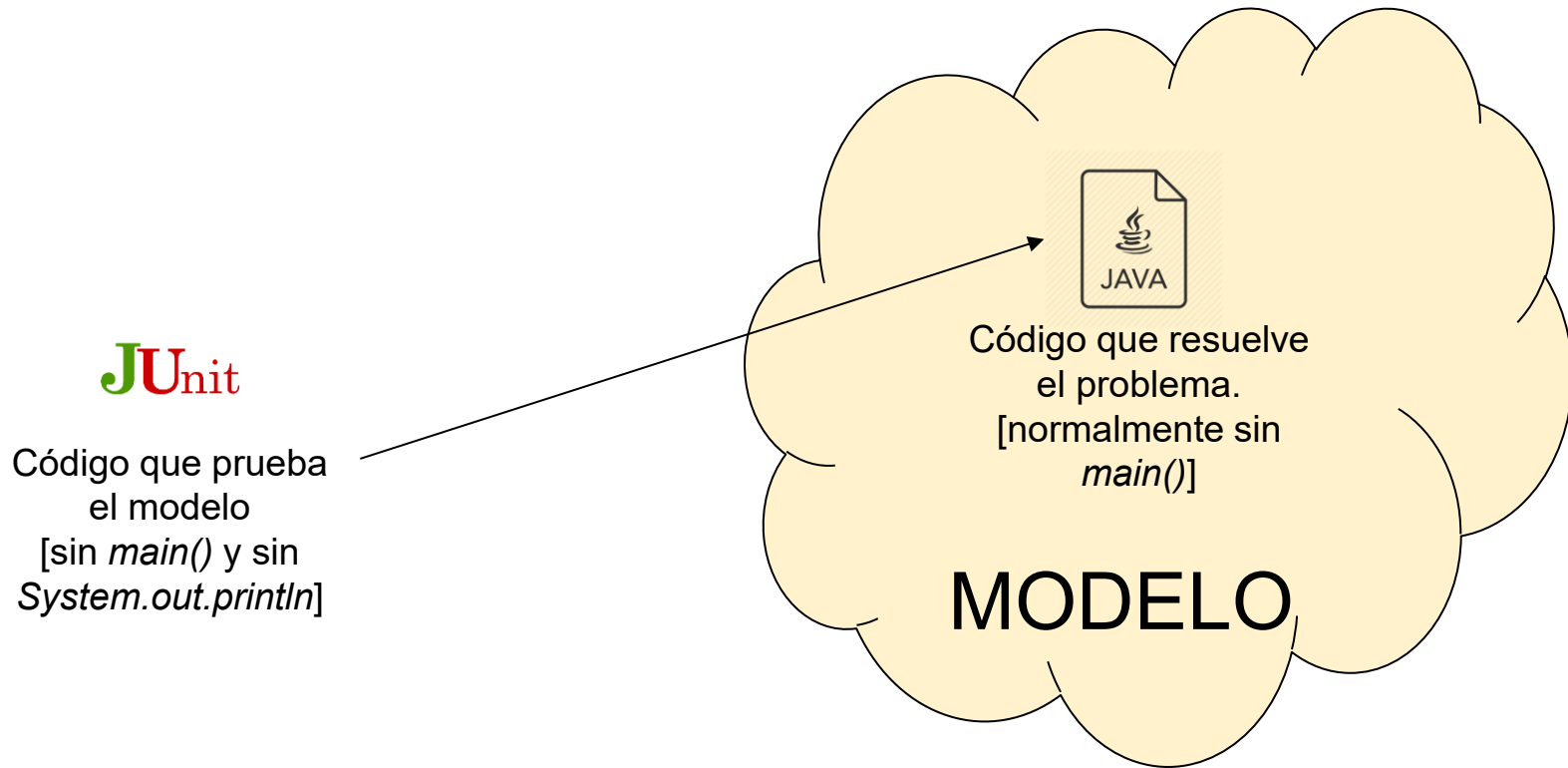
¿Cómo lo habíamos resuelto hasta ahora?



¿Cómo lo habíamos resuelto hasta ahora?



Usaremos JUnit para probar nuestro código



Clase de pruebas

- Encontrar errores de programación en cada clase ejecutando sus métodos
- Con combinaciones de parámetros y atributos que pueden descubrir algún error
 - Casos normales, casos extremos, casos raros...
- Por cada clase bajo pruebas, creamos una clase de prueba
- Por cada método de la clase (quitando los obvios), creamos al menos un método de prueba en la clase de prueba

Método de pruebas

- Cada método de prueba:
 - Crea un objeto de la clase bajo pruebas
 - Pone valores en los atributos y parámetros necesarios
 - Llama al método bajo prueba y guarda el resultado obtenido
 - Compara el resultado esperado (lo que daría si estuviera bien) con el resultado obtenido
 - Si son compatibles está bien; si no, falla y se indica el fallo (mensaje por la pantalla)

- Biblioteca Java para facilitar la escritura y ejecución de las pruebas (en eclipse)
- Crear una clase de prueba (TestComplejo) a partir de una clase bajo pruebas (Complejo)
 - New->Other->Java->JUnit->JUnit Test Case
- Crear esqueletos de métodos de prueba a partir de los métodos bajo pruebas

```
@Test // indicación de que es un método de prueba
public void testModulo() {
    fail("Not yet implemented"); // siempre falla
}
```

Junit: aserciones

- Métodos de Junit: comparan el valor esperado con el valor obtenido –si no son compatibles se indica el fallo con mensaje (opcional)
- **assertTrue** (boolean comprobación, String msg)
 - Éxito si la comprobación es cierta (falsa)
- **assertEquals** (Objeto esperado, Objeto obtenido, String msg)
 - Éxito si objetos iguales con equals
- **assertEquals** (double esperado, double real, double e, String msg)
 - Éxito si $\text{Math.abs}(\text{esperado} - \text{real}) \leq e$
- **assertSame** (Objeto esperado, Objeto obtenido, String msg)
 - Éxito si referencias iguales empleando == (o distintas)
- **assertNull** (Objeto objeto, String msg)
 - Éxito si el objeto es nulo
- **fail** (String msg)
 - La prueba falla siempre
- **assertFalse**, **assertNotEquals**, **assertNotSame**, **assertNotNull**

Métodos de prueba especiales (JUnit4)

- El método bajo pruebas lanza excepción (falla si no lanza la excepción o lanza otra):

- **@Test con el tipo de excepción esperada:**

```
@Test(expected=ImporteNoDisponibleException.class)
public void testreintegroExcesivo () throws
    ImporteNoDisponibleException {
    Cuenta c = new Cuenta("Jose Perez", 1000);
    c.reintegro(1001);
}
```

- El método bajo pruebas puede demorarse mucho por error (la prueba falla si pasan 10 milisegundos)

```
@Test (timeout=10)
public void testTimeout() {
    Cuenta c = new Cuenta ("José Pérez", 1000);
    c.bucleInfinito();
}
```

Métodos de prueba especiales (JUnit5)

- El método bajo pruebas lanza excepción (falla si no lanza la excepción o lanza otra):

```
@Test
public void testreintegroExcesivo () {
    assertThrows(ImporteNoDisponibleException.class, () -> {
        Cuenta c = new Cuenta("Jose Perez", 1000);
        c.reintegro(1001);
    });
}
```

- El método bajo pruebas puede demorarse mucho por error (la prueba falla si pasan 10 milisegundos)

```
@Test
public void testTimeout() {
    assertTimeout(Duration.ofMillis(1), () -> Thread.sleep(10));
}
```

TestComplejo

```
import static org.junit.Assert.*;
import org.junit.*;

public class TestComplejo {
    private static final double delta = 0.00001;

    @BeforeEach
    public void setUp(){
        // Optativo: para inicializar cada método de prueba
    }

    @AfterEach
    public void tearDown(){
        // Optativo: para finalizar cada método de prueba
    }
}
```

```
@Test
public void testOpuesto() {
    Complejo c = new Complejo(0,0);

    assertEquals(-0.0, c.opuesto().getI(), delta, "Falla X");
    assertEquals(-0.0, c.opuesto().getR(), delta, "Falla Y");
}
```

Ejecución de pruebas

- Sobre la clase de prueba: Run As-> Junit Test

